Visualização Geométrica, Rasterização e Recorte de Árvores Arteriais CCO

Brenda Gabrielle Alves Nascimento[1] (24.1.4011)

[1]Departamento de Computação (DECOM), Universidade Federal de Ouro Preto (UFOP)

BCC327 – Computação Gráfica, 2026.2 | Prof. Dr. Rafael Alves Bonfim de Queiroz

RESUMO

Contexto: Árvores arteriais sintéticas geradas pelo método *Constrained Constructive Optimization* (CCO) são descritas por pontos, segmentos e o raio de cada vaso, e precisam ser convertidas em imagem para análise visual. **Objetivo:** Desenvolver uma aplicação de Computação Gráfica que visualize os modelos CCO 2D e 3D e demonstre, com implementação própria, os algoritmos clássicos envolvidos na construção da imagem. **Metodologia:** A aplicação, em Python com pygame-ce, PyOpenGL e NumPy, tem um visualizador com leitor VTK, mapas de cores, faixas e tubos proporcionais ao raio, matrizes próprias de transformação, câmera e projeção, e MSAA; e um modo didático cujo framebuffer é escrito apenas pelos algoritmos do grupo: Bresenham, DDA, Scanline Fill, supersampling, Cohen–Sutherland, Liang–Barsky, Sutherland–Hodgman e Weiler–Atherton. **Resultados:** Os 12 modelos foram lidos corretamente; as matrizes coincidem com as funções equivalentes do OpenGL; os algoritmos foram validados contra testes de referência sem divergências; e as comparações visuais evidenciam projeções, aliasing e os limites do Sutherland–Hodgman com polígonos côncavos. **Conclusão:** A separação entre visualizador e modo didático permitiu aliar uma visualização fluida dos modelos à demonstração transparente, pixel a pixel, de cada algoritmo.

Palavras-Chave: Computação Gráfica; Método CCO; Rasterização; Recorte; Antialiasing.

1 Introdução

O método CCO, introduzido por Schreiner e Buxbaum [1], gera árvores vasculares sintéticas que suprem um domínio de perfusão sob restrições fisiológicas. Cada modelo é um grafo em árvore: pontos no espaço, segmentos que ligam pares de pontos e um raio associado a cada segmento. Para que esses modelos sejam inspecionados, é preciso transformá-los em imagem, o que envolve todas as etapas clássicas do *pipeline* gráfico.

Este trabalho desenvolve uma aplicação para visualizar, transformar, rasterizar e recortar os modelos CCO 2D e 3D fornecidos em formato VTK. O foco não é apenas usar uma biblioteca gráfica, mas implementar e demonstrar os algoritmos clássicos. As contribuições são:

- um leitor VTK próprio e um visualizador com linhas, faixas 2D e tubos 3D codificados por cor e espessura em função do raio;
- matrizes próprias de translação, rotação, escala, câmera e projeções ortográfica e perspectiva, validadas contra o OpenGL;
- um modo didático com framebuffer próprio para Bresenham, DDA, Scanline Fill e supersampling;
- implementação e comparação de Cohen–Sutherland, Liang–Barsky, Sutherland–Hodgman e, como diferencial, Weiler–Atherton.

2 Fundamentação Teórica

2.1 Modelos CCO e formato VTK

Os modelos estão no formato VTK legado ASCII, com `DATASET POLYDATA`. A seção `POINTS` lista as coordenadas (x, y, z); a seção `LINES` lista as células, cada uma no formato `2 i j`, ligando o ponto i

ao ponto j; e a seção `CELL_DATA` traz o escalar `raio`, um valor por segmento. Como o raio é atributo de célula, toda a cor e a espessura de um vaso derivam de um único valor. As coordenadas estão em metros (domínio com cerca de 5 cm de raio) e o raio em milímetros.

2.2 Modelo de cor

Um mapa contínuo de cores associa a cada escalar normalizado $t = (r - r_{\min})/(r_{\max} - r_{\min}) \in [0, 1]$ uma cor RGB obtida por interpolação linear entre cores de controle. Foram usados dois mapas. O *jet* (arco-íris) reproduz o estilo das figuras de referência e tem alto contraste, mas não é perceptualmente uniforme: o ciano e o amarelo criam fronteiras aparentes inexistentes nos dados [10]. O *viridis* é perceptualmente uniforme e legível por daltônicos e em escala de cinza. O modelo RGB foi adotado por ser o espaço nativo do framebuffer.

2.3 Transformações geométricas e câmera

Em coordenadas homogêneas, um ponto $\mathbf{p} = (x, y, z, 1)^T$ é transformado por matrizes 4×4, o que permite expressar translação como produto matricial:

$$T = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad S = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad R_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \tag{1}$$

O vértice percorre $\mathbf{p}_{\text{recorte}} = P \cdot V \cdot M \cdot \mathbf{p}_{\text{modelo}}$, em que M leva o modelo ao **espaço do mundo**, V leva o mundo ao **sistema de coordenadas da câmera** e P aplica a projeção. O espaço do mundo é o referencial fixo da cena, comum a todos os objetos. O sistema da câmera é a mesma cena descrita do ponto de vista do observador: a câmera fica na origem, olhando para $-z$. A matriz V é montada a partir do olho $\mathbf{e}$, do alvo $\mathbf{a}$ e de um vetor "cima" $\mathbf{c}$, que definem a base ortonormal da câmera

$$\mathbf{w} = \frac{\mathbf{e} - \mathbf{a}}{\|\mathbf{e} - \mathbf{a}\|}, \qquad \mathbf{u} = \frac{\mathbf{c} \times \mathbf{w}}{\|\mathbf{c} \times \mathbf{w}\|}, \qquad \mathbf{v} = \mathbf{w} \times \mathbf{u}, \tag{2}$$

e V translada o olho para a origem e reescreve cada ponto nessa base.

Na **projeção ortográfica** os raios de projeção são paralelos: a matriz apenas mapeia uma caixa no cubo $[-1, 1]^3$, e o tamanho aparente não depende da distância. Na **projeção perspectiva**, a última linha da matriz é $(0, 0, -1, 0)$, que copia a profundidade para w; na divisão por w, objetos distantes ficam menores e retas paralelas convergem.

2.4 Rasterização de segmentos

O algoritmo de Bresenham [4] rasteriza um segmento usando apenas aritmética inteira. No caso $0 \le m \le 1$, x avança sempre uma unidade e a variável de decisão p_k, proporcional à diferença entre as distâncias da reta ideal aos dois pixels candidatos, escolhe entre manter ou incrementar y:

$$p_0 = 2\Delta y - \Delta x, \qquad p_{k+1} = \begin{cases} p_k + 2\Delta y, & p_k < 0 \\ p_k + 2\Delta y - 2\Delta x \quad (y \leftarrow y + 1), & p_k \ge 0. \end{cases} \tag{3}$$

Os demais octantes são obtidos trocando os papéis de x e y e os sinais dos incrementos. O DDA, implementado para comparação, soma incrementos reais $\Delta y/\Delta x$ e arredonda cada posição, exigindo ponto flutuante a cada passo.

2.5 Preenchimento de polígonos por Scanline

O Scanline Fill [2] percorre o polígono linha a linha. Uma *tabela de arestas* (ET) guarda cada aresta não horizontal no balde da linha onde começa, com $(y_{\max}, x, 1/m)$. Em cada linha, as arestas que começam entram na *tabela de arestas ativas* (AET), as que terminaram saem, os x são ordenados e preenchidos aos pares (regra par-ímpar), e cada x avança $1/m$ para a linha seguinte, aproveitando a coerência entre linhas. O intervalo semiaberto $[y_{\min}, y_{\max})$ faz com que um vértice compartilhado seja contado uma única vez.

2.6 Aliasing e antialiasing

O aliasing surge porque um pixel só pode estar aceso ou apagado: bordas inclinadas viram degraus e detalhes menores que um pixel geram padrões espúrios. No **supersampling** (SSAA), a cena é renderizada numa grade k vezes maior em cada direção e reduzida pela média de cada bloco $k \times k$, de modo que a cor final é proporcional à cobertura. No **MSAA**, cada pixel armazena várias amostras de cobertura e profundidade, mas o fragmento é sombreado uma única vez; na resolução, as amostras são promediadas. O custo é muito menor que o do SSAA e o efeito se concentra nas bordas dos objetos.

2.7 Recorte de segmentos

O **Cohen–Sutherland** [5] atribui a cada extremidade um código de 4 bits que indica em quais semiplanos externos à janela o ponto está. Se $c_0 \vee c_1 = 0$, o segmento é aceito inteiro; se $c_0 \wedge c_1 \neq 0$, as duas extremidades estão do mesmo lado de fora e o segmento é rejeitado; caso contrário, uma extremidade externa é substituída pela interseção com uma borda e o teste se repete. O **Liang–Barsky** [6] usa a forma paramétrica $P(t) = P_0 + t(P_1 - P_0)$, $t \in [0, 1]$, e as desigualdades $p_k t \leq q_k$, com $p = (-\Delta x, \Delta x, -\Delta y, \Delta y)$ e $q = (x_0 - x_{\min}, x_{\max} - x_0, y_0 - y_{\min}, y_{\max} - y_0)$. Para $p_k < 0$ a reta entra pela borda e $t_0 = \max(t_0, q_k/p_k)$; para $p_k > 0$ ela sai e $t_1 = \min(t_1, q_k/p_k)$; se $t_0 > t_1$, nada resta. As interseções só são calculadas no final.

O **Cyrus–Beck** [7], apenas discutido, generaliza a ideia paramétrica para *qualquer janela convexa*: para cada aresta, com normal $\mathbf{n}_i$ e ponto $\mathbf{f}_i$, resolve $\mathbf{n}_i \cdot (P(t) - \mathbf{f}_i) = 0$ e classifica a interseção como entrada ou saída pelo sinal de $\mathbf{n}_i \cdot (P_1 - P_0)$. Sua principal diferença em relação aos dois anteriores é não exigir janela retangular alinhada aos eixos; o Liang–Barsky é justamente o Cyrus–Beck especializado para esse caso, em que os produtos escalares se reduzem aos p_k e q_k.

2.8 Recorte de polígonos

O **Sutherland–Hodgman** [8] trata a janela convexa como interseção de semiplanos e recorta o polígono contra cada aresta em sequência. Para cada aresta $S \to P$ do polígono: se ambos estão dentro, emite P; se só S está dentro, emite a interseção I; se ambos estão fora, nada; se só P está dentro, emite I e P. Ele sempre produz *um único* polígono; quando o resultado correto tem várias componentes, elas ficam ligadas por arestas degeneradas sobre a borda da janela. O **Weiler–Atherton** [9] insere as interseções nas listas de vértices do sujeito e da janela, classifica-as como entrada ou saída e percorre as duas fronteiras alternadamente, produzindo uma lista de polígonos. Ele é mais adequado quando o sujeito ou a janela são côncavos, quando o resultado tem múltiplas regiões ou quando há interseções complexas entre as fronteiras.

3 Metodologia e Arquitetura da Aplicação

A aplicação foi escrita em Python 3 com pygame-ce 2.5.2 (janela e entrada), PyOpenGL 3.1.7 e NumPy 2.1.3. Ela é composta por dois programas que compartilham os módulos da Tabela 1: o **visualizador**, que usa OpenGL para desenhar os modelos com as matrizes e a geometria calculadas pelo grupo, e o **modo didático**, uma matriz NumPy de 80×60 pixels, ampliada na tela, na qual apenas os algoritmos

do grupo escrevem. O pygame-ce somente amplia essa matriz, de modo que nenhum pixel do modo didático é produzido pela biblioteca.

Tabela 1: Módulos da aplicação e origem de cada funcionalidade.

Módulo	**Conteúdo**	**Origem**
`leitor_vtk.py`	Leitor de VTK legado (POINTS, LINES, CELL_DATA)	grupo
`cores.py`	Mapas jet e viridis por interpolação linear	grupo
`geometria.py`	Faixas 2D (retângulos e discos) e tubos 3D (cilindros, esferas e normais)	grupo
`transformacoes.py`	Translação, rotação, escala, câmera, projeções	grupo
`rasterizacao.py`	Bresenham, DDA e Scanline Fill	grupo
`antialiasing.py`	Supersampling com filtro caixa	grupo
`recorte.py`	Cohen–Sutherland, Liang–Barsky, Sutherland–Hodgman, Weiler–Atherton	grupo
`visualizador.py`	Rasterização dos triângulos, profundidade, iluminação, MSAA, VBOs	OpenGL
`modo_didatico.py`	Framebuffer próprio e interações; janela e texto	grupo / pygame-ce

Primitivas. Além de linhas, cada segmento 2D vira uma faixa de largura $2r$ (dois triângulos), com um disco em cada extremidade que arredonda as junções. Em 3D, cada segmento vira um cilindro de 12 lados construído a partir da base ortonormal $(\mathbf{d}, \mathbf{e}_1, \mathbf{e}_2)$ do segmento, com esferas nos pontos da árvore; cada esfera recebe o raio e a cor do maior vaso que passa pelo ponto. A espessura desenhada é $r \times 10^{-3} \times$ exagero, convertendo milímetros em metros, com exagero ajustável (padrão $2\times$). Os vasos são desenhados do mais fino para o mais grosso, para que os troncos fiquem por cima nas bifurcações. A iluminação dos tubos (luz direcional e ambiente do pipeline fixo) é da API e serve apenas para dar volume; o estudo de iluminação é tema do TP2.

Transformações e observador. A matriz do modelo é $M = T \cdot R_x \cdot R_y \cdot R_z \cdot S \cdot N$, em que N centraliza o modelo e normaliza seu tamanho para que a mesma câmera sirva a 2D e 3D. Há quatro vistas (frontal, lateral, superior e isométrica). Para que o modelo não mude de tamanho ao trocar de projeção, a caixa ortográfica tem meia altura $h = d\tan(\text{fovy}/2)$, igual à altura do tronco de pirâmide perspectivo na distância d do alvo. Os eixos do mundo são desenhados só com V, permanecendo fixos enquanto o modelo é transformado.

Antialiasing. O SSAA do modo didático reutiliza o Bresenham e o Scanline: a cena é rasterizada numa grade k vezes maior, com cada pixel de linha carimbado como um bloco $k \times k$ (para manter a largura de um pixel final), e reduzida com `reshape` e média. A mesma função com $k = 1$ gera a versão serrilhada, garantindo uma comparação justa. O MSAA do visualizador usa um framebuffer fora da tela (FBO) com 4 amostras por pixel, resolvido para a janela com `glBlitFramebuffer`, o que não depende de a janela oferecer multiamostragem.

Recorte. Os segmentos são recortados em coordenadas reais e só depois rasterizados, o que caracteriza recorte geométrico e não apenas descarte de pixels. O Sutherland–Hodgman aceita qualquer janela convexa, com o teste "dentro" feito pelo sinal do produto vetorial $(B - A) \times (P - A)$ para a janela em sentido anti-horário. No Weiler–Atherton, vértices exatamente sobre arestas, comuns em uma grade inteira, impedem a classificação de entrada e saída; nesses casos a janela é deslocada por 10^{-6} vezes o tamanho da cena, valor desprezível para o resultado.

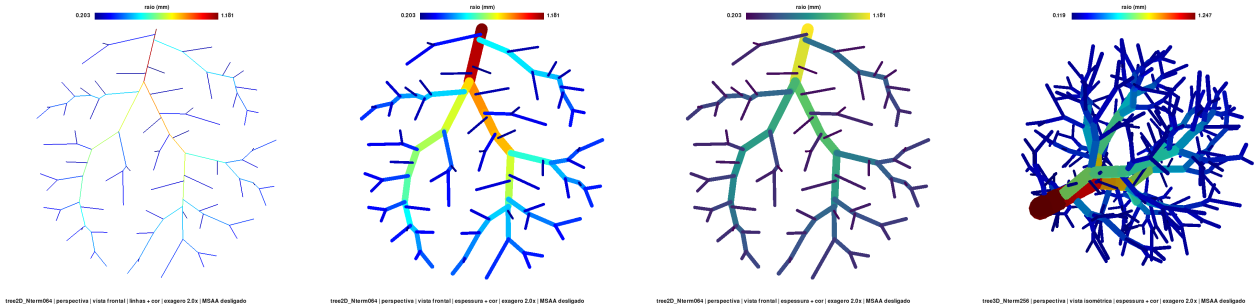
4 Resultados

4.1 Leitura e representação dos modelos

A Tabela 2 resume os modelos lidos. Em todos, o número de pontos é $2N_{\text{term}}$ e o de segmentos é igual ao de pontos menos um, assinatura de uma árvore conexa e sem ciclos, o que confirma a leitura. A Figura 1 mostra as representações do visualizador.

Tabela 2: Modelos CCO lidos: número de pontos, segmentos e intervalo do raio.

Modelo 2D	**Pontos**	**Seg.**	$r_{\min}$	$r_{\max}$	**Modelo 3D**	**Pontos**	**Seg.**	$r_{\min}$	$r_{\max}$
Nterm008	16	15	0,2491	0,6668	Nterm016	32	31	0,2261	0,6870
Nterm016	32	31	0,2279	0,8176	Nterm032	64	63	0,2132	0,8373
Nterm032	64	63	0,2058	0,9804	Nterm064	128	127	0,2052	1,0178
Nterm064	128	127	0,2032	1,1814	Nterm128	256	255	0,1454	1,2258
Nterm128	256	255	0,1454	1,1930	Nterm256	512	511	0,1192	1,2472
Nterm256	512	511	0,1074	1,1937	Nterm512	1024	1023	0,0956	1,2666

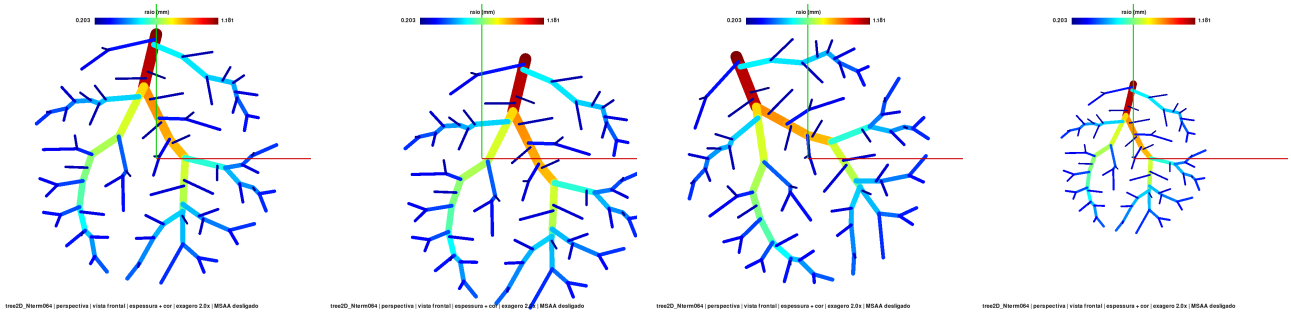


(a) linhas, jet (b) faixas, jet (c) faixas, viridis (d) tubos 3D

Figura 1: Representações do modelo: (a) linhas coloridas pelo raio; (b) e (c) espessura e cor proporcionais ao raio nos dois mapas; (d) tubos 3D iluminados (Nterm256).

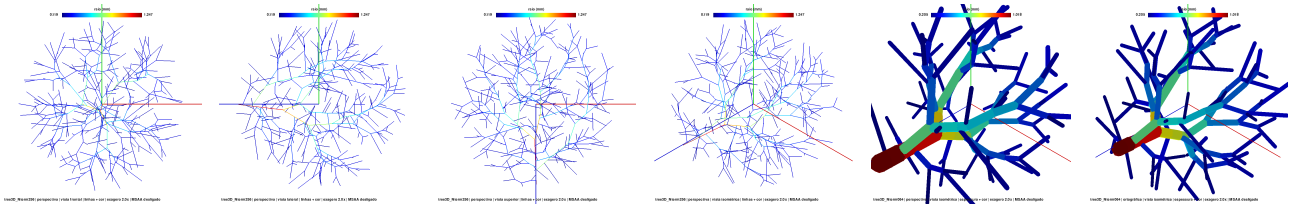
4.2 Transformações, observador e projeções

A Figura 2 mostra translação, rotação e escala aplicadas ao modelo, com os eixos do mundo fixos. A Figura 3 mostra as quatro vistas do observador e a comparação entre projeções. Na perspectiva, o tronco próximo à câmera aparece mais largo que os ramos distantes de mesmo raio; na ortográfica, a espessura aparente não depende da profundidade. As matrizes próprias foram comparadas elemento a elemento com `glOrtho`, `gluPerspective`, `gluLookAt` e `glRotatef`, coincidindo em todos os casos (tolerância 10^{-5}).



(a) original (b) translação (c) rotação de 35° em z (d) escala 0,6

Figura 2: Transformações geométricas aplicadas ao modelo. Os eixos do mundo (x em vermelho, y em verde) usam apenas a matriz de visão e permanecem fixos.



(a) frontal (b) lateral (c) superior (d) isométrica (e) perspectiva (f) ortográfica

Figura 3: (a) a (d): quatro posições do observador para o modelo 3D Nterm256. (e) e (f): mesma câmera com projeção perspectiva e ortográfica (Nterm064).

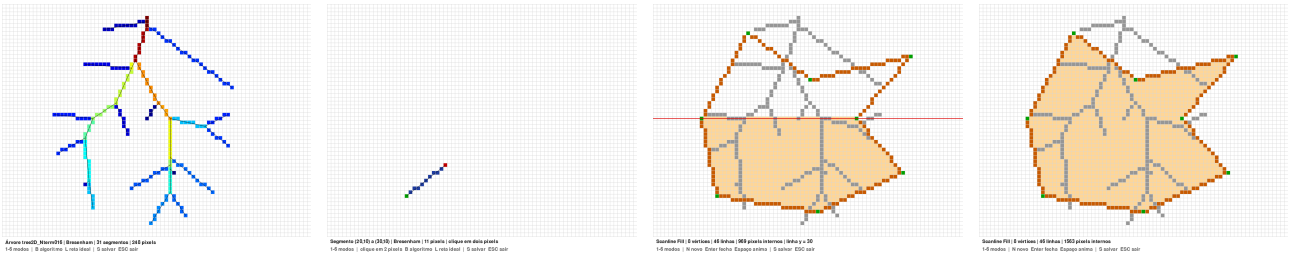
4.3 Rasterização de linhas e polígonos

A Tabela 3 traz o exemplo numérico do Bresenham para o segmento $(20, 10)$–$(30, 18)$, com $\Delta x = 10$ e $\Delta y = 8$. Nos passos com $p_k < 0$ (passos 2 e 7), y é mantido no passo seguinte. Em 20 000 segmentos aleatórios, a implementação sempre começou e terminou nas extremidades, gerou pixels 8-conexos e manteve todos a no máximo meio pixel da reta ideal; o DDA divergiu em cerca de 17% dos casos, sempre em empates de arredondamento, que são igualmente válidos.

Tabela 3: Exemplo numérico do Bresenham de $(20, 10)$ a $(30, 18)$: $p_0 = 6$, $2\Delta y = 16$, $2\Delta y - 2\Delta x = -4$.

k	0	1	2	3	4	5	6	7	8	9	10
p_k	6	2	−2	14	10	6	2	−2	14	10	6
x	20	21	22	23	24	25	26	27	28	29	30
y	10	11	12	12	13	14	15	16	16	17	18

Para o Scanline Fill, o polígono côncavo $(2, 1), (12, 3), (9, 7), (12, 11), (5, 9), (2, 11)$ gera, na linha $y = 10$, quatro interseções (2,00; 3,50; 8,50; 11,25) e dois trechos, [2..3] e [9..11], correspondentes à reentrância. Comparado a um teste par-ímpar ponto a ponto em 400 polígonos aleatórios, não houve divergências. A Figura 4 mostra a árvore rasterizada, o exemplo numérico e o preenchimento de uma região de interesse.

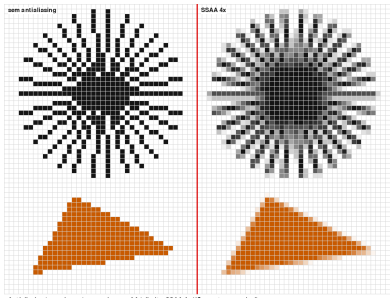


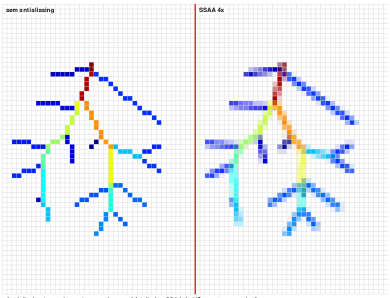
(a) árvore, Bresenham (b) segmento da Tab. 3 (c) Scanline em andamento (d) Scanline completo

Figura 4: Modo didático: (a) segmentos da árvore Nterm016 com a reta ideal sobreposta; (b) pixels do exemplo numérico; (c) e (d) região de interesse côncava preenchida por Scanline, com a linha de varredura em vermelho.

4.4 Aliasing e antialiasing

A Figura 5 compara a mesma cena sem antialiasing e com SSAA 4× (16 amostras por pixel), e o efeito do MSAA 4× na borda de um tubo. No leque de linhas, a região central, em que as linhas ficam mais próximas que um pixel, vira um padrão quadriculado sem antialiasing e uma transição suave com SSAA.


(a) leque: sem AA × SSAA 4×


(b) árvore: sem AA × SSAA 4×


(c) sem MSAA


(d) MSAA 4×

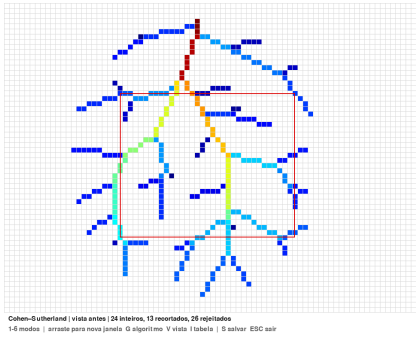
Figura 5: (a) e (b): supersampling próprio, com a metade esquerda serrilhada e a direita suavizada. (c) e (d): borda de um tubo no visualizador, ampliada 3×.

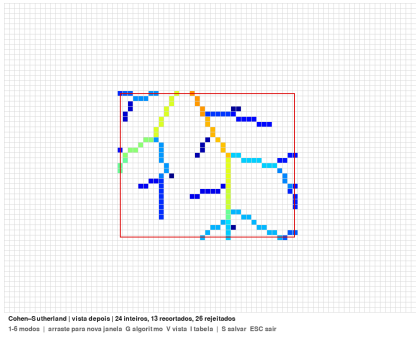
4.5 Recorte de segmentos

A Tabela 4 mostra o recorte do segmento $(2, 6)$–$(36, 30)$ pela janela $[10, 30] \times [10, 25]$. O Cohen–Sutherland calcula a interseção (7,67; 10), que ainda está fora da janela (código `0001`), e precisa de uma nova interseção. O Liang–Barsky calcula quatro razões q/p e apenas os dois pontos finais. Ambos chegam a (10; 11,65)–(28,92; 25). Em 100 000 segmentos aleatórios em $[-3, 3]^2$ com janela $[-1, 1]^2$, os resultados foram idênticos (50 810 recortados, 47 910 rejeitados e 1 280 aceitos inteiros). A Figura 6 mostra o recorte da árvore.

Tabela 4: Recorte do segmento $(2, 6)$–$(36, 30)$ pela janela $[10, 30] \times [10, 25]$.

	Cohen–Sutherland		**Liang–Barsky**				
Passo	P_0 [código]	P_1 [código]	**Borda**	p	q	t_0	t_1
0	(2,00; 6,00) [0101]	(36,00; 30,00) [1010]	esquerda	−34	−8	0,2353	1,0000
1	(7,67; 10,00) [0001]	(36,00; 30,00) [1010]	direita	34	28	0,2353	0,8235
2	(10,00; 11,65) [0000]	(36,00; 30,00) [1010]	baixo	−24	−4	0,2353	0,8235
3	(10,00; 11,65) [0000]	(28,92; 25,00) [0000]	cima	24	19	0,2353	0,7917


(a) antes


(b) depois

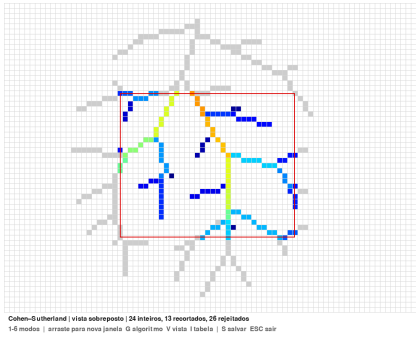

(c) sobreposto

Figura 6: Recorte dos segmentos da árvore Nterm032 pela janela em vermelho: 24 segmentos aceitos inteiros, 13 recortados e 26 rejeitados.

4.6 Recorte de polígonos

No exemplo do Sutherland–Hodgman com o pentágono $(5, 15), (20, 3), (35, 18), (22, 32), (18, 20)$ e a janela $[10, 30] \times [10, 25]$, o polígono passa de 5 para 6, 7, 8 e 9 vértices após cada aresta da janela. A Figura 7 mostra os dois casos que diferenciam os algoritmos. Na forma em U, cuja base fica fora da janela, o Sutherland–Hodgman devolve um polígono com uma ponte degenerada (em magenta), enquanto o Weiler–Atherton devolve dois polígonos (cores distintas). Com uma janela côncava em L,

o Sutherland–Hodgman recorta pela interseção dos semiplanos das arestas e perde parte da região, enquanto o Weiler–Atherton produz o resultado correto. Em 600 pares de polígonos simples, côncavos e com casos degenerados, o Weiler–Atherton não apresentou divergências em 150 000 pontos de teste.



(a) U, Sutherland–Hodgman (b) U, Weiler–Atherton (c) janela em L, S–H (d) janela em L, W–A

Figura 7: Recorte de polígonos. (a) ponte degenerada ligando as duas componentes; (b) duas componentes separadas; (c) resultado incorreto do Sutherland–Hodgman com janela côncava; (d) resultado correto do Weiler–Atherton.

5 Discussão

Comparação entre algoritmos de recorte de segmentos. O Liang–Barsky evita interseções desperdiçadas e calcula apenas os pontos finais, enquanto o Cohen–Sutherland pode calcular interseções ainda externas, como no exemplo da Tabela 4. Em compensação, a rejeição trivial do Cohen–Sutherland custa apenas operações sobre bits. No experimento com 100 000 segmentos, em que quase metade foi rejeitada trivialmente, o Cohen–Sutherland foi mais rápido em Python (cerca de 0,7 μs contra 1,1 μs por segmento). A vantagem relativa depende, portanto, da proporção de segmentos totalmente fora da janela. O Cyrus–Beck seria necessário apenas para janelas convexas não retangulares.

Sutherland–Hodgman e Weiler–Atherton. O Sutherland–Hodgman é simples e eficiente, e o resultado preenchido por Scanline fica correto mesmo com pontes, porque elas têm área nula e ficam sobre linhas horizontais ou verticais. Porém o contorno exibe arestas espúrias e a saída não representa componentes separadas, o que é inadequado quando o resultado alimenta outras etapas. O Weiler–Atherton resolve esses casos e aceita janelas côncavas, ao custo de estruturas de dados mais complexas e de tratamento explícito de casos degenerados.

Decisões de implementação. A espessura foi modelada como geometria (faixas e tubos) e não com `glLineWidth`, que é limitado pelo hardware e aplica uma única largura por chamada. Os vértices são enviados uma única vez para a GPU em VBOs, pois o modelo Nterm512 gera 90 088 triângulos. O MSAA foi feito por FBO porque o servidor gráfico do WSL não ofereceu janela multiamostrada.

Limitações. No ambiente de desenvolvimento (WSL com renderização por software), o modelo Nterm512 em tubos fica com taxa de quadros baixa durante a rotação. A iluminação dos tubos é da API. O deslocamento da janela no Weiler–Atherton resolve os casos degenerados encontrados, mas é uma solução numérica, não simbólica. O SSAA usa filtro caixa, que é o mais simples; filtros com pesos (como o gaussiano) suavizariam melhor.

6 Conclusão

O trabalho cumpriu os objetivos do TP1: os modelos CCO 2D e 3D foram lidos e representados por linhas, faixas e tubos codificados por cor e espessura; as transformações, as vistas e as projeções foram implementadas com matrizes próprias; e os algoritmos de rasterização, preenchimento, antialiasing e recorte foram implementados, validados e comparados, incluindo o Weiler–Atherton como diferencial. A separação entre o visualizador e o modo didático permitiu demonstrar cada algoritmo pixel a pixel sem depender da biblioteca. Como continuidade, o TP2 deve incorporar visibilidade, iluminação própria, transparência, textura e técnicas baseadas em raios sobre essa mesma base.

Referências

[1] SCHREINER, W.; BUXBAUM, P. F. Computer-optimization of vascular trees. **IEEE Transactions on Biomedical Engineering**, v. 40, n. 5, p. 482–491, 1993.

[2] FOLEY, J. D.; VAN DAM, A.; FEINER, S. K.; HUGHES, J. F. **Computer Graphics: Principles and Practice**. 2. ed. Reading: Addison-Wesley, 1990.

[3] HEARN, D.; BAKER, M. P. **Computer Graphics, C Version**. 2. ed. Upper Saddle River: Prentice Hall, 1997.

[4] BRESENHAM, J. E. Algorithm for computer control of a digital plotter. **IBM Systems Journal**, v. 4, n. 1, p. 25–30, 1965.

[5] NEWMAN, W. M.; SPROULL, R. F. **Principles of Interactive Computer Graphics**. 2. ed. New York: McGraw-Hill, 1979.

[6] LIANG, Y.-D.; BARSKY, B. A. A new concept and method for line clipping. **ACM Transactions on Graphics**, v. 3, n. 1, p. 1–22, 1984.

[7] CYRUS, M.; BECK, J. Generalized two- and three-dimensional clipping. **Computers & Graphics**, v. 3, n. 1, p. 23–28, 1978.

[8] SUTHERLAND, I. E.; HODGMAN, G. W. Reentrant polygon clipping. **Communications of the ACM**, v. 17, n. 1, p. 32–42, 1974.

[9] WEILER, K.; ATHERTON, P. Hidden surface removal using polygon area sorting. **ACM SIGGRAPH Computer Graphics**, v. 11, n. 2, p. 214–222, 1977.

[10] BORLAND, D.; TAYLOR II, R. M. Rainbow color map (still) considered harmful. **IEEE Computer Graphics and Applications**, v. 27, n. 2, p. 14–17, 2007.